Review Annex D for C++26

Recommendations to remove or undeprecate specific features

Document #:	P2863R5
Date:	2024-04-16
Project:	Programming Language C++
Audience:	EWG, LEWG

Reply-to:	Alisdair Meredith <ameredith1@bloomberg.net>

Contents

§ 1 Abstract

This paper evaluates all the deprecated facilities of the C++23 Standard and recommends removing a subset from Annex D in C++26, either by removal from the Standard entirely or by undeprecation and restoring the subset to the main text. Such recommendations will be pursued in specific papers, and this paper acts as an index into that work.

§ 2 Revision History

§ R5: April 2024 (post-Tokyo mailing)

- Updated status of tracked papers following Tokyo meeting reviews and polls
  - P2865: SG22
  - P2866: CORE -> (needs update), LWG -> (needs Annex C)
  - P2867: LWG -> WG21 -> WP
  - P2869: LWG -> WG21 -> WP
  - P2872: POLL -> LWG -> WG21 -> WP
  - P2873: SG16 -> LEWG
  - P2875: POLL -> LWG -> WG21 -> WP
  - P3047: LEWG
  - P2863: (to confirm no action)
    - D.22: LEWG
    - D.29: SG16

§ R4: February 2024 (midterm mailing)

- Verified all reviews from the Kona 2023 meeting are recorded
- Confirmed completed status (DONE) for all papers that landed in working draft N4971
- Updated results from the December LEWG electronic polling, [P3054R0]
- Updated status of tracked papers following reviews and polls

- P2864: WP -> DONE
- P2865: SG22
- P2866: POLL -> CORE/LWG
- P2867: POLL -> LWG
- P2868: WP -> DONE
- P2869: POLL -> LWG
- P2870: WP -> DONE
- P2871: WP -> DONE
- P2872: LEWG -> POLL
- P2873: SG16
- P2875: LEWG -> POLL
- P3047: LEWG (new paper)
- P2863: (confirm no action)
  - D.22: LEWG
  - D.29: SG16

R3: December 2023 (post-Kona)

- Tentatively completed EWG review process
- Reengaged SG16 review process
- Remove dated revision when tracking papers still making progress
- Cite dated revision for papers that pass a plenary poll
- Updated status of tracked papers following reviews and polls
  - P2864: DONE -> EWG -> CORE -> WP
  - P2865: CORE -> EWG -> SG22
  - P2866: LEWG -> POLL
  - P2867: LEWG -> POLL
  - P2868: POLL -> LWG -> WP
  - P2869: LEWG -> POLL
  - P2870: POLL -> LWG -> WP
  - P2871: POLL -> LWG -> WP
  - P2872: No progress
  - P2873: LEWG -> SG16
  - P2875: No progress
  - P2984: EWG -> DONE
  - P3047: LEWG (pending new paper)
  - P2863: (confirm no action)
    - D.6 : EWG -> P2984R0 -> DONE
    - D.12: LEWG -> DONE
    - D.13: LEWG -> DONE
    - D.14: LEWG -> LEWG (Paper requested)
    - D.16: LEWG -> DONE
    - D.19: LEWG -> DONE
    - D.22: Skipped by mistake
    - D.23: Defer to [P3039R0] by David Stone

- D.29: LEWG -> SG16

R2: October 2023 (pre-Kona)

- Added revision numbers to the revision history
- Updated revision numbers of papers being tracked for progress
- Updated status of tracked papers
  - P2865: EWG -> CORE
  - P2871: LEWG -> POLL
  - P2874: WP -> DONE
  - P2984: EWG -> EWG (new paper)

R1: August 2023 (post-Varna update)

- Completed SG1 review process
- Completed SG16 review process
- Completed initial EWG review process
- Completed initial LWG review process
- Provided a clearer rationale for taking no action on D.29
- Fixed links to Core and LWG issues
- Updated C++23 reference doc to [N4950]
- Added new progress states:
  - POLL : LEWG Electronic polling
  - WG21 : Ready for a plenary poll
  - WP : Approved for WP, not yet published
  - DONE : All anticipated committee work complete
- Updated status of tracked papers following initial reviews
  - P2864: EWG -> DONE, no action
  - P2865: EWG -> CORE -> EWG
  - P2866: SG1 -> LEWG, EWG -> LEWG
  - P2867: Not reviewed
  - P2868: LEWG -> POLL
  - P2869: SG1 -> LEWG
  - P2870: LEWG -> POLL
  - P2871: SG16 -> LEWG
  - P2872: LEWG -> SG16 -> LEWG
  - P2873: SG16 -> LEWG
  - P2874: LWG -> WG21 -> WP
  - P2875: LEWG -> LEWG (requested update)
  - P2863: (confirm no action)
    - D.3 : EWG -> DONE, no action
    - D.6 : EWG -> EWG, request paper to undeprecate
    - D.7 : EWG -> DONE, no action
    - D.8 : EWG -> DONE, no action
    - D.9 : EWG -> DONE, no action
    - D.10: EWG -> DONE, no action
    - D.12: Not reviewed

- D.13: Not reviewed
- D.14: Not reviewed
- D.16: Not reviewed
- D.19: Not reviewed
- D.22: Not reviewed
- D.23: Not reviewed
- D.29: SG16 -> LEWG

R0: May 2023 (pre-Varna mailing)

- Initial draft of this paper.

3 Outline

With the release of a new C++ Standard, we get an opportunity to revisit the features identified for deprecation, and consider if we are prepared to clear any out yet, either by removing completely from the standard, or by reversing the deprecation decision and restoring the feature to full service. This paper makes no attempt to offer proposals for removing features other than those deprecated in Annex D, nor does it attempt to identify new candidates for deprecation.

In an ideal world, the start of every release cycle would cleanse the list of deprecated features entirely, allowing the language and library to evolve cleanly without holding too much dead-weight. It should be noted that only two of the current twenty-nine deprecated feature were deprecated prior to C++17, as we have made good progress are resolving long-standing deprecations. However, the desire to support customers and not needlessly break their code makes a full clear-out impractical. This paper continues the tradition of maintaining a single paper to focus attention of efficiently reviewing the case for and against removal, or undeprecation, of each deprecated feature of the just-published standard, the meeting after that standard has been sent to ISO for the final publication ballot.

In contrast to previous years, all work to change the status of a feature, by removal or undeprecation, will be delegated to a paper on that feature. Thus, work proceeds on each feature without delaying everything to the pace of the slowest feature and the author's ability to update this summary paper in a timely manner. Note that many reviews will still be processed by this paper, where no further action is intended on a feature for C++26.

The benefits of making the choice to remove features early in a standard cycle is that we will get the most experience we can from the bleeding-edge adopters whether a particular removal is more problematic than expected - but even this data point is limited, as bleeding-edge adopters typically have less reliance on deprecated features, eagerly adopting the newer replacement facilities.

However, do note that with the three year release cadence for the C++ Standard, we will often be re-evaluating features whose deprecated status has barely reached print.

We have precedent that Core language features are good targets for removal, typically taking two standard cycles to remove a deprecated feature, although often prepared to entirely remove a feature even without a period of deprecation, if the cause is strong enough.

The library experience has been mixed, with no desire to remove anything without a period of deprecation (other than `gets`) and no precedent prior to C++17 for actually removing deprecated features.

Informal feedback on library deprecation when performing these reviews for previous standards leans in two directions: deprecation is for life, and we should never remove anything, as there is no reason for the library to ever break client code; and alternatively with the addition of zombie names there is clear intent library vendors should continue supporting deprecated features as long as their customers demand them, without violating conformance, and that removal from the standard removes maintenance costs from wg21 (placed instead on library vendors) and that this paper should be aggressive in its recommendations to remove deprecated features.

4 Methodology

We will review each deprecated facility, and make a recommendation to either remove that feature, undeprecate that feature, or take no action. A recommendation to remove or undeprecate will delegate that work to a follow-up paper, tracked by a checklist below. The author provides several papers that do just that for features where the abandoned review for C++23 had expressed a strong intent to remove. In other cases "Request Paper" is indicated, which would be the result of evolution review wanting to proceed with undeprecation or removal that the current author does not have the resources to pursue.

§ 5 Checklist For Recommendations

This table will track progress of all papers targeting features deprecated in the C++23 Standard, according to the last public draft, [N4950]. As such, the subclause references will not update if new deprecations occur during C++26, as we track only the features deprecated in the current published standard.

It notes the standard that introduced the feature; the standard and paper that deprecated that feature; the current recommendation of action to take, with a reference to the delegated paper that will propose those changes; and the current working group that has ownership of that review. Any recommendation of "No action" means the review is in this paper.

Once the review of a feature is completed, either by confirming No action or adopting a reviewed paper in plenary, the Owner will change to Done.

§ 5.1 Core

Subclause	Feature	Introduced	Deprecated	By Paper	Recommendation	Owner
D.2	Arithmetic conversion on enumerations	C++98	C++20	[P1120R0]	Remove: [P2864R2]	DONE
D.3	Implicit capture of `*this` by reference	C++11	C++20	[P0806R2]	No action	DONE
D.4	Array comparisons	C++98	C++20	[P1120R0]	Remove: [P2865]	SG22
D.5	Deprecated use of `volatile`	C++98	C++20	[P1152R4]	Address by [P2866]	CORE
D.6	Redeclare static `constexpr` members	C++11	C++17	[P0386R2]	No action	DONE
D.7	Non-local use of TU-local entities	C++98	C++20	[P1815R2]	No action	DONE
D.8	Implicit special members	C++98	C++11	[N3203]	No action	DONE
D.9	Some literal operator declarations	C++11	C++23	[CWG2521]	No action	DONE
D.10	`template` keyword before qualified names	C++98	C++23	[P1787R6]	No action	DONE

§ 5.2 Library

Subclause	Feature	Introduced	Deprecated	By Paper	Recommendation	Owner
D.11	*Requires:* clauses	C++98	C++20	Editorial	Remove: [P2874R1]	DONE
D.12	`has_denorm` members in `numeric_limits`	C++98	C++23	[P2614R2]	No action	DONE
D.13	Deprecated C macros	C++98	C++23	[P2790R0]	Defer to rebase on C23	DONE
D.14	`relops`	C++98	C++20	[P0768R1]	Remove: [P3047]	LEWG

Subclause	Feature	Introduced	Deprecated	By Paper	Recommendation	Owner
D.15	`char *` streams	C++98	C++98	N/A	Remove: [P2867R2]	WP
D.16	Deprecated error numbers	C++11	C++23	[P2790R0]	No action	DONE
D.17	The default allocator	C++17	C++23	[LWG3170]	Remove: [P2868R3]	DONE
D.18	`polymorphic_allocator::destroy`	C++17	C++23	[LWG3036]	Undeprecate: [P2875R4]	WP
D.19	Deprecated type traits	C++11	C++20	[P0767R1]	No Action	DONE
		C++11	C++23	[P1413R3]		
D.20	`volatile` tuple API	C++11	C++20	[P1831R1]	Remove: [P2866]	CORE
D.21	`volatile` variant API	C++17	C++20	[P1831R1]	Remove: [P2866]	CORE
D.22	`std::iterator`	C++98	C++17	[P0174R2]	No action	LEWG
D.23	`move_iterator::operator->`	C++11	C++20	[P1252R2]	Defer to [P3039R0]	DONE
D.24	C API to use `shared_ptr` atomically	C++11	C++20	[P0718R2]	Remove: [P2869R4]	WP
D.25	`basic_string::reserve()`	C++98	C++20	[P0966R1]	Remove: [P2870R3]	DONE
D.26	`<codecvt>`	C++11	C++17	[P0618R0]	Remove: [P2871R3]	DONE
D.27	`wstring_convert` et al.	C++11	C++17	[P0618R0]	Remove: [P2872R3]	WP
D.28	Deprecated locale category facets	C++11	C++20	[P0482R6]	Remove: [P2873]	LEWG
D.29	`filesystem::u8path`	C++17	C++20	[P0482R6]	No action	SG16
D.30	atomic operations	C++11	C++20	[P0883R2]	Address by [P2866]	LWG

§ 6 Review By Clause

§ 6.1 General [depr.general]

There is no normative content in the general clause, so nothing to do. We mention it here only so that the automatic subtitle numbering stays in sync with Annex D of [N4950].

§ 6.2 Arithmetic conversion on enumerations [depr.arith.conv.enum]

Implicit arithmetic conversion on unscoped enumerations when involved in mixed operations with another type are part of our C heritage, but were deprecated to match up with the design of the spaceship operator for C++20 by paper [P1120R0].

A full rationale for why we should act to remove these deprecated conversions now, along with full core wording, is addressed by paper [P2864R2].

§ 6.2.1 EWG Review, 2023 June, Varna

Concerns were raised about a lack of real implementation experience, especially in SFINAE contexts where deprecation warnings cannot reach. There remain some concerns about C compatibility. See [P2864R2] for details.

There will be no further progress on this topic for C++26, and we should look to present a stronger motivation and usage experience for the C++29 review.

§ 6.2.2 EWG Review, 2023 November, Kona

§ 6.2.3 Mailing, December 2023

Adopted into Working Draft N4971.

§ 6.3 Implicit capture of `*this` by reference [depr.capture.this]

This feature was deprecated in C++20 by [P0806R2]. Its removal would potentially impact on programs written against C++11 or a later standard, when lambda captures were first introduced.

The concern addressed by the paper is that the implicit capture of `this` as a reference to data members on a default-capture that copies is surprising and often misleading. C++20 introduced the explicit capture of `this` as a pointer, or `*this` by value, to clearly disambiguate the different use cases.

MSVC, gcc, and EDG front ends have been warning about this deprecation since their experimental C++20 support, before the standard itself was ratified. As of the publication of this paper, the Clang compiler still does not warn on use of this feature, although a patch to add that warning is checked into the compiler trunk for what should become Clang 17.0.

Experience over the last few years has been mixed, as several compilers would warn on the preferred syntax as redundant *before* C++20, so it becomes difficult to get a warning free build with a single syntax, and is typically achieved by users proving a macro to choose the preferred form.

The lack of warning in Clang suggests the may be many users of modern C++ that are still unaware of this deprecation, so the tentative recommendation is to take no action in C++26, and reconsider more carefully for C++29. However, the mixed experience of having users decide which warning they prefer suggests we might want to take more decisive action for this review, which would require a follow-up paper to consider the merits of removal vs. undeprecation.

§ 6.3.1 EWG Review, 2023 June, Varna

Consensus to leave deprecated for another release cycle.

§ 6.4 Array comparisons [depr.array.comp]

Implicit array-to-pointer decay when invoking any of the comparison operators is part of our C heritage, but these conversions were deprecated to match up with the design of the spaceship operator for C++20 by paper [P1120R0].

The implicit decay in these cases is often misleading and rarely helpful. A full rationale for why we should act to remove these deprecated conversions now, along with full core wording, is addressed by paper [P2865].

§ 6.4.1 EWG Review, 2023 June, Varna

No concerns were raised, and this paper proceeds to Core.

§ 6.4.2 CWG Review, 2023 June, Varna

Concerns were raised about comparison with null pointers, and this paper was sent back to EWG.

§ 6.4.3 Mailing, October 2023

Wording updated to preserve non-deprecated semantics. Created paper [P2984R0] to consider deprecation and removal of array comparison with null pointer constants.

Paper sent back to Core.

§ 6.5 Deprecated `volatile` types [depr.volatile.type]

The `volatile` keyword is part of our C heritage, but a variety of usages were deprecated for C++20, in conjunction with our C liaison, by paper [P1152R4].

As the C committee looked to adopt our deprecations, they got feedback from their vendors that they considered some of the deprecated uses as essential, so a subset of this functionality was undeprecated for C++23 by [P2327R1].

The remaining set of deprecated operations remain an often misleading source of bugs, so paper [P2866] provides further rationale for why we should act to remove these deprecated conversions now, along with full core and library wording.

§ 6.5.1 EWG Review, 2023 June, Varna

Extensive discussions about volatile structured bindings, see [P2866] for details.

Ultimately, there was consensus to send this document onto its next stage, which is LEWG, and then proceeding straight to Core.

§ 6.6 Redeclaration of `static constexpr` data members [depr.static.constexpr]

Static `constexpr` data members were added to the language in C++11, as part of the initial `constexpr` feature. However, they still required a definition of the member outside the class. When inline variables were added in C++17 by [P0386R2] then `static constexpr` data members became implicitly `inline`, and the external definition became redundant, so was deprecated. By the time C++26 is published, such out-of-class definitions will have been deprecated longer than they were required.

However, as of writing this paper in March 2023, none of the 4 major compiler front ends report a deprecated use warning on the example in the standard, including the latest trunk build for open source compilers. Therefore, the tentative recommendation of this paper is for no action in the standard towards removing this feature, and encourage the front ends to start reporting deprecated usage to their users. We might consider undeprecation given the lack of enthusiasm of vendors to warn on redundant definitions, although that would feel like a step backwards, maintaining an exception to the One Definition Rule.

When considering the removal of redundant definitions, it seems simple enough to support a code base that is common with C++11 and the feature removal (or warned deprecation) with a simple feature check:

```
#if !defined(__cpp_inline_variables)
constexpr Type Class::Member;
#endif
```

§ 6.6.1 EWG Review, 2023 June, Varna

There is no interest in removing a feature that does no harm, and that no compilers are yet issuing deprecation warnings for.

Then the question was whether to undeprecate this feature, as deprecation seems to have had no impact.

Poll: EWG is interested in undeprecating defining inline `constexpr` class variables (P2863R0 section 6.6).

```
SF  F  N  A SA
 2 12  7  3  0
```

A follow-up paper is expected from the same author promoting and proposing undeprecation.

§ 6.6.2 Mailing, October 2023

Submitted paper [P2984R0] to address this deprecation.

§ 6.7 Non-local use of TU-local entities [depr.local]

This feature was deprecated for C++20 as part of the effort to cleanly introduce modules into the language, and was adopted by paper [P1815R2]. It potentially impacts on code written against C++98 and later standards.

This feature was deprecated only at the final meeting of the C++20 development cycle, and at the time this paper is written, deployment experience with standard modules is still limited, and it is not clear if we have any insight into how much code has been impacted by this deprecation. No current compilers issue a deprecation warning on affected code. As such, the tentative recommendation is too take no action — there is not even sufficient experience to consider undeprecation.

§ 6.7.1 EWG Review, 2023 June, Varna

Consensus to leave deprecated for another release cycle.

§ 6.8 Implicit declaration of copy functions [depr.impldec]

This feature was deprecated for C++11 by paper [N3203] as part of integrating a good user experience defining classes now that rvalue references are part of the language. The deprecated parts address the default behavior of C++03 code.

The following test program will demonstrate deprecation warnings in gcc since release 9.1, and Clang since release 10.0. For both compilers, the command line switch `-Wextra` is required to enable these deprecation warnings. As of the publication of this paper in April 2023, the EDG and MSVC front ends do not appear to support this warning yet.

```
struct A {
    A() = default;
    A(A const&){}
};

struct B {
    B& operator=(B const&){ return *this; }
};

struct C {
```

```
    ~C() {}
};

int main() {
    A a{};
    A ax = a;
    a = ax;     // use of deprecated operator

    B b{};
    B bx = b;   // use of deprecated constructor
    b = bx;

    C c{};
    C cx = c;   // no-one warns on deprecated constructor
    c = cx;     // no-one warns on deprecated operator
}
```

Note that no compiler is warning on copy operations in case `C`, declaring a user provided destructor.

This topic was last considered by EWG for C++23 at the 2022 Kona meeting, where core issue 2132 looked into undeprecation. The notes on that discussion offer nothing more than an immediate call for consensus to not deprecate, and close that issue as NAD.

This is the oldest deprecated Core language facility, the only one remaining that was deprecated before C++17. When C++26 is published it will have been deprecated for 16 years so the tentative recommendation for this paper is to request authors for a paper providing a considered analysis of removal and undeprecation, in whole or in part, for C++26.

§ 6.8.1 EWG Review, 2023 June, Varna

Consensus to leave deprecated for another release cycle. We would consider re-opening if someone came forward with a paper that addressed user impact in the real world, especially with SFINAE contexts that are not easily detected by deprecation warnings.

§ 6.9 Literal operator function declarations using an identifier [depr.lit]

The use of whitespace between the `operator ""` and the following identifier to denote a user-defined literal suffix was deprecated for C++23 by Core issue 2521. In deprecating this feature at the 2023 Issaquah meeting, there is a clear intent that this is one step along the way to actively remove that support:

EWG had consensus on “The form of User Defined Literals that permits a space between the quotes and the name of the literal should be deprecated, and eventually removed. Additionally, the UDL name should be excluded from the restriction in 5.10 [lex.name] in the non-deprecated form (sans space).”

Given the lateness in the process of this deprecation, no currently available compiler has a deprecation warning, nor even the trunk builds of open source compilers. Hence, the tentative recommendation is to take no action.

However, given the expressed intent to actively remove the support for that whitespace, we may want to consider how actively we want to pursue removal, and request a paper to research whether such removal would be viable in the 3 year release cycle from C++23 to C++26.

§ 6.9.1 EWG Review, 2023 June, Varna

Consensus to leave deprecated for another release cycle.

§ 6.10 `template` keyword before qualified names [depr.template.template]

This feature was deprecated for C++23 by paper [P1787R6], Declarations and where to find them. The following code sample is used to test whether compilers have implemented that paper yet, and whether they diagnose the deprecated use:

```
template <class T> struct A {
   void f(int);
   template <class U> void f(U);
};

template <class T>
struct B {
   template <class T2> struct C { };
};

// deprecated: T::C is assumed to name a class template:
template < class T
```

```
         , template <class X> class TT = T::template C
         >
struct D { };
D<B<int> > db;

// recommended: T::C is assumed to name a class template:
template < class T
         , template <class X> class TT = T::C
         >
struct E { };
E<B<int> > db;
```

Testing with the latest compilers available from Godbolt Compiler Explorer shows that no current compiler has implemented this part of that paper yet. Without the implementation, not only are there no deprecation warnings, the recommended code transformation does not compile.

Given the current lack of implementation, it seems far too early to consider removing this feature, so the tentative recommendation is to take no action.

Given the change in C++11 to accept redundant use of the `typename` keyword in Core issue 382 we might consider undeprecating this feature before compilers start issuing deprecation warning.

Further, C++11 also allowed redundant use of `::template` where not required in non-template cases, Core issue 468, suggesting a level of redundancy is desirable so that users are not expected to have such a precise mental compiler, especially when learning the language.

§ 6.10.1 EWG Review, 2023 June, Varna

Consensus to leave deprecated for another release cycle.

§ 6.11 Requires paragraph depr.res.on.required

This style of documentation was deprecated editorially for C++20 following the application of a sequence of papers to update each main library clause, consistently following the new conventions established by paper [P0788R3]. The author provides a paper with tentative wording to apply the last of those changes to Annex D, [P2874R1].

Note that resolving this will touch wording in clauses D.14, D.19, D.24, D.27, and D.29. We do not track these changes in the paper index above, as they have no impact on whether deprecated library facilities are removed, although the edits would be helpful should we desire to undeprecate any of those clauses.

§ 6.11.1 EWG Review, 2023 June, Varna

After applying corrections, sent [P2874R1] to poll in the Varna plenary.

§ 6.11.2 Mailing, October 2023

Adopted into Working Draft N4958.

§ 6.12 `has_denorm` members in `numeric_limits` [depr.numeric.limits.has.denorm]

This small part of `numeric_limits` for floating-point types was deprecated for C++23 by paper [P2614R2].

As a relatively late change to the Standard Library, we have little experience in how widely triggered the deprecation warning will be, so the tentative recommendation of this paper is to take no action.

However, it is worth noting that the Zombie Names policy means that even if we were to remove this feature from the standard tomorrow, vendors can continue to maintain this feature as a conforming extension for as long as their customers demand, so with the perceived low risk it may be worth asking for a paper to remove this feature entirely from C++26.

§ 6.12.1 Initial LEWG Review: Kona, 2023/11/07

It was observed that code searches show almost no use of these names. However, they were deprecated only within the last 12 months as part of the C++23 standard, that is still awaiting the final stages of publication.

Consensus to make no changes for C++26.

§ 6.13 Deprecated C macros [depr.c.macros]

The C Standard Library headers were undeprecated for C++23 by paper [P2340R1]. Then the C macros that report that identifiers corresponding to C++ keywords have been defined as macros in language support headers were again deprecated as "immediate issues" by paper [P2790R0], resolving national body comments.

It was noted during review that these tokens will become true keywords in C23, so the pending C Standard will be removing these macros. Hence, they remain deprecated in C++23, and we except to see them removed by a paper updating the C++26 Standard to use the latest C Standard Library.

The tentative recommendation of this paper is to take no action as part of this deprecation review, anticipating the C library update paper where this change would be one small part of the larger whole.

§ 6.13.1 Initial LEWG Review: Kona, 2023/11/07

Consensus to follow the suggestion of this paper, and handle any progress on removal as part of any future paper to rebase our standard onto C23.

§ 6.14 Relational operators [depr.relops]

The `std::rel_ops` namespace was introduced in the original C++ Standard, so its removal would potentially impact on code written against C++98 and later standards. It was deprecated with the introduction of support for the 3-way comparison "spaceship" operator in C++20, by paper [P0768R1].

As of publishing this paper, none of the three popular standard libraries give any deprecation warnings on the sample code in the Tony tables below.

Deprecated

```
#include <cassert>
#include <utility>

struct Test {
   int data = 0;

friend bool operator==(Test a, Test b){return a.data ==
        b.data;}
friend bool operator <(Test a, Test b){return a.data <
        b.data;}
};

int main() {
    Test x{};
    Test y{2};

    using namespace std::rel_ops;
    assert(x == x);
    assert(x != y);

    assert(x <  y);
    assert(x <= y);
    assert(x >= y);
    assert(x >  y);
}
```

Modern

```
#include <cassert>
#include <compare>

struct Test {
   int data = 0;

friend bool operator ==(Test, Test) =
        default;
friend auto
        operator<=>(Test, Test) =
        default;
};

int main() {
    Test x{};
    Test y{2};

    // No using namespace
    assert(x == x);
    assert(x != y);

    assert(x <  y);
    assert(x <= y);
    assert(x >= y);
    assert(x >  y);
}
```

The 2023 version of this paper erroneously claimed that all the comparisons would be synthesized from the same two operations relied upon by the `rel_ops` operators. In fact, those rules rely upon supplying the 3-way comparison operator, rather than `operator<`, as illustrated above.

Note the three changes:

- `#include` a different header
- provide a different operator
  - note that definitions can be defaulted now
- do not rely on a using directive

Given the current lack of deprecation warnings, the tentative recommendation of this paper is to take no action for C++26 and encourage Standard Library maintainers to annotate their implementations as deprecated before the next standard cycle.

As the `using namespace std::rel_ops` idiom may be seen as encouraging bad hygiene, especially when applied at global/namespace scope in a header, there may still be folks motivated to write a paper expressing a stronger intent to remove this feature for C++26.

As a historical note, the Standard Library specification itself used to rely on `std::rel_ops` to provide the specification for any comparison operator if the necessary wording were missing. However, as part of C++20, it was confirmed that no current wording relies on that legacy fall-back, and the corresponding as-if wording was removed.

6.15 `char *` streams [depr.str.strstreams]

The `char*` streams were provided, pre-deprecated, in C++98 and have been considered for removal before. All the necessary facilities to migrate to safer and easier to use streaming facilities were added in C++20 and C++23, so the recommendation is to remove the deprecated `char *` streams from C++26 by [P2867R2].

6.15.1 LEWG Reflector Review : 2023/10/02 – 2023/10/09

Forward to POLL with 20 votes for and none against.

6.15.2 LEWG electronic poll : 2023/12/20 – 2023/01/10

Poll taken: [P3053R0]

Poll results: [P3054R0]

Conclusion: Forward to LWG

6.16 Deprecated error numbers [depr.cerrno]

Several macros and enumerators for `enum class errc` were deprecated for C++23 by [P2790R0]. While there is no apparent urgency for their removal given how recently they were deprecated, the zombie names clause would also give vendors adequate coverage to retain support at their discretion. This proposal weakly recommends retaining these names until C++29.

6.16.1 Initial LEWG Review: Kona, 2023/11/07

These names were deprecated late in the C++23 process, and that standard is still working its way through ISO publication. Consensus to make no changes here for C++26, allowing time to gain experience with the deprecation.

6.17 The default allocator [depr.default.allocator]

The Standard Library `allocator` class has a member that can be synthesized from the primary `allocator_traits` template, and was deprecated by [#LWG3170]. By providing this member directly, any classes that derive from `std::allocator` will not synthesize this value correctly, but use the `true_type` value provided directly by `std::allocator`, forcing such allocators to provide their own override when that value could otherwise by synthesized correctly.

While this is a small corner for misuse, the concern is embarrassing to explain, and the Standard Library allocator is a common example folks will follow when trying to write their first allocators. Hence, this paper recommends the removal of this deprecated typedef for C++26 by [P2868R3].

6.17.1 LEWG Reflector Review : 2023/07/25 – 2023/08/02

Discussion led to an updated paper with more rationale.

Final tally: 10 forward to POLL, no objections.

Forward to the next electronic poll to send to LWG.

6.17.2 LEWG electronic poll : 2023/09/21 – 2023/10/10

Poll taken: [P2972R0]

Poll results: [P3020R0]

Conclusion: Forward to LWG

6.17.3 Mailing, December 2023

Adopted into Working Draft N4971.

§ 6.18 Deprecated `polymorphic_allocator` member function [depr.mem.poly.allocator.mem]

This feature was deprecated by [#LWG3036]. However, the author of this paper believes that `std::pmr::polymorphic_allocator` is an allocator that will be used in non-generic circumstances, unlike `std::allocator`, so this member function that could otherwise be synthesized should still be part of its pubic interface. Hence, the recommendation is to undeprecate with paper [P2875R4]

§ 6.18.1 LEWG Reflector Review : July 2023

Paper withdrawn until it presents more rationale.

Revised paper available in this mailing.

§ 6.19 Deprecated type traits [depr.meta.types]

The `is_pod` trait was deprecated for C++20 by paper [P0767R1] as part of removing the POD vocabulary from the C++ Standard, both core and library. The term had changed meaning so frequently that it no longer served as useful vocabulary. The type trait was extracted to Annex D, and now itself provides the only definition in the standard for a POD. Client code is encouraged to use the more specific traits for trivial and standard layout types to better describe their need. Note that the related term POF, for Plain Old Function, was removed from C++17 by paper [P0270R3].

The `is_pod` trait was first supplied as part of C++11, so its removal would potentially impact programs written against the C++11 Standard or later. Users have had up to three years of implementations warning on use of the deprecated trait, so we could consider removal, with the usual proviso that the name is preserved as a zombie for previous standardization. As the case for removal is not urgent, this paper recommends the removal of this trait from the C++26 Standard, but only weakly. However, the current wording does not follow library best practices, and should be updated to better specify the Requires clauses with our modern vocabulary if it is retained.

The type traits `aligned_storage` and `aligned_union` were deprecated for C++23 by [P1413R3]. They do use modern library wording, and as they do no active harm the recommendation is to retain them for C++26 to allow proper time for users to update their code, and consider again for removal in C++29.

§ 6.19.1 Deployment

Tested the following program to observe deprecation warnings for the `is_pod` trait through a variety of standard library implementations at Godbolt Compiler Explorer:

```
#include <type_traits>

int main() {
   static_assert(std::is_pod<int>::value, "oops");
   static_assert(std::is_pod_v<int>, "oops");
}
```

- libc++ No deprecation warning
- libstdc++ gcc 10.1
- Microsoft No deprecation warning

§ 6.19.2 Initial LEWG Review: Kona, 2023/11/07

Most of these type traits were deprecated only as recently as C++23, which is still wending its way through the ISO publication process. The remaining trait has been retained as the one remaining place in the standard that defines and uses the term POD. Consensus to make no changes for C++26

§ 6.20 Tuple [depr.tuple]

This library was deprecated as part of the work on deprecating unnecessary volatile facilities, so its removal is recommended by paper [P2866].

§ 6.20.1 Initial LEWG Review: Kona, 2023/11/07

TBD

§ 6.20.2 LEWG electronic poll : 2023/12/20 – 2023/01/10

Poll taken: [P3053R0]

Poll results: [P3054R0]

Conclusion: Forward to LWG

§ 6.21 Variant [depr.variant]

This library was deprecated as part of the work on deprecating unnecessary volatile facilities, so its removal is recommended by paper [P2866].

§ 6.21.1 Initial LEWG Review: Kona, 2023/11/07

TBD

§ 6.21.2 LEWG electronic poll : 2023/12/20 – 2023/01/10

Poll taken: [P3053R0]

Poll results: [P3054R0]

Conclusion: Forward to LWG

§ 6.22 Deprecated `iterator` class template [depr.iterator]

The class template `iterator` was first deprecated in C++17 by the paper [P0174R2]. The concern was that providing the needed support for iterator typenames through a templated base class, determining which name maps to which type purely by parameter order, was less clear than simply providing the needed names. Further, there were corner cases in usage that fell out of template syntax that made this tool hard to recommend as a simpler way of providing the type names, yet that was its whole reason to exist.

When this facility was reviewed for removal in C++20, it was noted that there were valid uses that relied on the default template arguments to deduce at least a few of the needed type names. Subsequent work on iterators and ranges ([P0896R4]) that landed in C++20 now means that work is also done by the primary `iterator_traits` template, and so the remaining use case (for new code) is also covered, making this class template strictly redundant.

The main concern that remains is breaking old code by removing this code from the standard libraries. That risk is ameliorated by the zombie names clause in the standard, allowing vendors to maintain their own support for as long as their customers demand. By the time C++23 ships, those customers will already have been on 6 years notice that their code might not be supported in future standards. However, we note the repeated use of the name `iterator` as a type within many containers means we might choose to leave this name off the zombie list. We conservatively place it there anyway, to ensure that we are covered by the previous standardization terminology to encompass uses other than as a container iterator typedef.

The recommendation of this paper is to take no action in C++26 until the is a stronger consensus for removal.

§ 6.22.1 Initial Review: telecon 2020/07/13

Concerns were raised about the lack of research into how much code is likely to break with the removal of this API. We would like to see more numbers and analysis on publicly available code, such as across all of Github. The better treatment of implicit generation of `iterator_traits` in C++23, and more familiarity with a limited number of code bases that still rely on this facility, gave more confidence in moving forward with removal than we had for C++20. It was also noted that the name may be unfortunate with the chosen form of concept naming adopted for C++20, and so its removal might lead to one fewer sources of future confusion. Given that implementers are likely to provide an implementation (through zombie names freedom) for some time after removal, there was consensus to proceed with removal, assuming the requested research does not reveal major concerns before the main LEWG review to follow.

§ 6.23 Deprecated `move_iterator` access [depr.move.iter.elem]

This feature was deprecated for C++20 by the paper [P1252R2] highlighting the concern that for a move iterator adapter, intending to expose its target as an rvalue (or xvalue), the arrow operator must return the original adapted iterator, which will likely produce an lvalue when dereferenced. This operator is not fit for purpose, and cannot be fixed. The workaround for users is to dereference the move iterator with `operator *` and call the member they wish to access using the familiar `.` notation. This preserves the value category of the iterator's target.

The proposal for C++20 was to deprecate this operator, with a view to removal at a later date. However, `operator->` support is part of the *Cpp17InputIterator* requirements, so cannot be removed without addressing that definition. One option might be to accept the `std::move_iterator` is *not* a *Cpp17InputIterator* at all, but models the `input_iterator` concept instead. That might mean deprecating the `iterator_category` typedef member as well.

Testing the following program with Godbolt Compiler Explorer, it appears that libc++ is the only Standard Library implementation, at the time of writing this paper, that warns on use of the deprecated operator, and has done so only since Clang 15:

```
#include <iterator>

struct Wrap {
    int data;
};

int main() {
   Wrap x = {42};
   std::move_iterator it = std::make_move_iterator(&x);
   auto y = it->data;
}
```

The recommendation is to take no action at this time, unless a more detailed paper is requested.

§ 6.23.1 Initial LEWG Review: Kona, 2023/11/07

It was observed that David Stone is doing work in this space with more support for implicitly generated operators, [P3039R0]. Consensus is to delegate this deprecated operator to that paper, thinking that is the appropriate direction to support removal of the deprecated function.

§ 6.24 Deprecated `shared_ptr` atomic access [depr.util.smartptr.shared.atomic]

The legacy C-style atomic API for manipulating shared pointers provided in C++11 is subtle, frequently misunderstood, and easily misused to cause data races. A type-safe replacement facility that also provides support for `atomic<weak_ptr<T>>` was added to C++20, so we recommend removing the legacy API at the earliest opportunity.

See paper [P2869R4] for a more detailed discussion and proposed wording.

§ 6.24.1 SG1 (concurrency) Review, 2023 June, Varna

Polled after discussion of the header compatibility concern:

Poll: Remove deprecated `shared_ptr` atomic access APIs from C++26, with any of the library options listed in P2689?

```
SF  F  N  A SA
 2  4  1  1  0
```

Consensus to move this paper to LEWG to resolve the header design issue, and then continue on to LWG.

§ 6.24.2 Initial LEWG Review: Kona, 2023/11/07

TBD

§ 6.24.3 LEWG electronic poll : 2023/12/20 – 2023/01/10

Poll taken: [P3053R0]

Poll results: [P3054R0]

Conclusion: Forward to LWG

§ 6.25 Deprecated `basic_string` capacity [depr.string.capacity]

See paper [P2870R3] for a more detailed discussion and proposed wording to remove this facility from C++26.

§ 6.25.1 LEWG Reflector Review : 2023/06/26 – 2023/07/02

There was one objection that was withdrawn when it was realized that the C++03 behavior became a no-operation in C++11, so finally removing that overload helps users find remaining code that needs updating to use the C++11 name to preserve behavior.

Final tally: 18 forward to POLL, no objections.

Forward to the next electronic poll to send to LWG.

6.25.2 LEWG electronic poll : 2023/09/21 – 2023/10/10

Poll taken: [P2972R0]

Poll results: [P3020R0]

Conclusion: Forward to LWG

6.25.3 Mailing, December 2023

Adopted into Working Draft N4971.

6.26 Deprecated standard code conversion facets [depr.locale.stdcvt]

This feature was originally proposed for C++11 by paper [N2007] and deprecated for C++17 by paper [P0618R0]. As noted at the time, the feature was underspecified, a source of security issues handling malformed Unicode, and would require more work than we wished to invest to bring it up to standard. Since then SG16 has been convened and is producing a steady stream of work to bring reliable well-specified Unicode support to C++.

Given vendors propensity to provide ongoing support for deprecated libraries under the zombie name reservations, we recommend removal from C++26, see paper [P2871R3] for details.

6.26.1 C++26 Review by SG16: telecon 2023/05/24

There were some concerns about removal before an alternative solution lands. However, it was then pointed out that the names in this API lie as they refer to UTF-16 but actually implement UCS-2, and so must provide a definition for UCS-2 as it is no longer defined in the Unicode standard.

No objections to removal in C++26.

6.26.2 LEWG Reflector Review : 2023/08/14 – 2023/09/15

Several folks who did not object were still concerned that we would be removing a facility before a replacement lands; others acknowledged they were aware of this when polling to move forward, and though this was still the right choice. If we did not know that library vendors would be relying on the Zombie Names clause, there may have been more objections.

Final tally: 13 forward to POLL, 1 Neutral-but-concerned.

Conclusion: forward to the next electronic poll to send to LWG.

6.26.3 LEWG electronic poll : 2023/09/21 – 2023/10/10

Poll taken: [P2972R0]

Poll results: [P3020R0]

Conclusion: Forward to LWG

6.26.4 Mailing, December 2023

Adopted into Working Draft N4971.

6.27 Deprecated convenience conversion interfaces [depr.conversions]

See paper [P2872R3] for a more detailed discussion and proposed wording to remove this facility from C++26.

6.27.1 C++26 Review by SG16: telecon 2023/05/24

SG16 reviewed P2872R0.

6.27.1.1 Summarize the following:

Giuseppe asked if the paper includes removal of `std::wbuffer_convert`.

Alisdair confirmed that it does.

Alisdair explained that these were deprecated because the example for `std::wstring_convert` used another deprecated feature, `std::codecvt_utf8` and, due to other underspecification concerns, no one was motivated to fix them.

Alisdair asked if SG16 is the right group to address this.

PBrett responded affirmatively and stated that SG16 is the group that misunderstands `wchar_t` the least.

Alisdair noticed some issues with the paper and concluded that updates are required before the paper is ready for any action to be taken on it.

§ 6.28 Deprecated locale category facets [depr.locale.category]

See paper [P2873] for a more detailed discussion and proposed wording to remove this facility from C++26.

§ 6.28.1 C++26 Review by SG16: telecon 2023/05/24

SG16 reviewed P2873R0.

§ 6.28.1.1 Summarize the following:

Tom explained that these facets were deprecated because they convert to and from UTF-8 in char-based storage rather than between the multibyte encoding like the non-deprecated facets do.

Tom reported that `char8_t`-based replacements were added as replacements, but those were a mistake because they won't be used by `char`-based streams anyway.

[Editor's note: [LWG3767] tracks deprecating the `char8_t`-based facets.]

PBrett asked for any objections to removal.

No objections were reported.

Corentin spoke in favor of removal.

§ 6.29 Deprecated filesystem path factory functions [depr.fs.path.factory]

A factory function to create path names from UTF-8 sequences of `char` was part of the original filesystem library adopted for C++17 ([P0218R1]). However, this was the only string-based factory function, as the preferred interface is to simply construct a path with a string of the corresponding type/encoding. This factory function was deprecated in C++20 with the addition of `char8_t` and the ability to now invoke a specific constructor for UTF-8 encoded (and typed) strings. See [P0482R6] for details.

The legacy API continues to function, but is more cumbersome than necessary. There appears to be no compelling case that the API is a risk through misuse, although the behavior is undefined if fed malformed UTF-8.

While it does no active harm, there is always a cost to maintaining text in the standard. The application of zombie names means that even if we remove this clause from Annex D in C++26, Standard Library vendors are likely to continue shipping to meet customer demand for some time to come. In the meantime, the current specification does not follow library wording best practices, and should be updated to better specify the *Requires:* clauses ([P2874R1]).

§ 6.29.1 C++23 Review for reference

This component was reviewed by telecon, achieving LEWG consensus for removal in C++23. However, the author ran out of time to complete the large paper handling all Annex D removals, and new information has since come to light with issue [LWG3840] requesting undeprecation of this function.

§ 6.29.2 LEWG telecon Review: 2023/01/10 (before R0 of this paper)

After some discussion there was little appetite for undeprecating `filesystem::u8path`. It is the only standard library interface that requires data provided in `char`-based storage to be encoded in a different character encoding than the execution character set or the encoding used for character and string literals.

For those that intentionally store UTF-8 data in `char`-based storage and would prefer not to use a deprecated interface, an equivalent to `filesystem::u8path` can be implemented in a few lines of code:

```
inline auto myu8path(const char* s) {
  std::u8string u8s(s, s+std::strlen(s));
  return std::filesystem::path(u8s);
}
```

SG16, the Unicode and Text Study Group, is evaluating approaches to enable restricted aliasing support for `char8_t` such that data in char-based storage could be passed to a `char8_t`-based interface without having to perform a copy as in the example code above; [P2626R0] is one such proposal.

6.29.3 C++26 Review by SG16: telecon 2023/05/24

SG16 observed that [P2626R0] aims to solve the root issue behind this problem, and that we should not remove a feature until it has co-existed at least one standard release with the facility to migrate to. There was also no enthusiasm for undeprecation, and a general leaning towards removal in due course, maybe C++29. They strongly recommend abiding by the status quo on this function for C++26.

6.29.4 Initial LEWG Review: Kona, 2023/11/07

Some folks mentioned using this interface, and how awkward it is to write code that is portable across multiple standards that does not rely on deprecated features; that problem would go away if we undeprecated this API, that behaves exactly as they expect.

Conversely, there is a desire to remove a potential source of confusion now that there is a type-safe interface that properly reflects the text encoding.

It was observed that it might be difficult to gain consensus in either direction — undeprecation or removal — and this feature may remain in limbo (Annex D) for many standards.

As the goal of this paper is to try to move deprecated features out of limbo, there was consensus to send this feature back to SG16 to consider their preferred direction for the long term future of this function.

6.30 Deprecated atomic operations [depr.atomics]

While it does no active harm, there is always a cost to maintaining text in the standard. This is similarly reflected in the C Standard, that initially deprecated the `ATOMIC_VAR_INT` macro (marked it as obsolescent) in C17, and is actively looking to remove it from the C2X Standard, per the paper WG14:N2390. We should strongly consider removing this macro, but perhaps as part of a broader paper to update our reference to the C23 Standard Library.

The original API to initialize atomic variables from C++11 was deprecated for C++20 when the atomic template was given a default constructor to do the right thing. See [P0883R2] for details.

The legacy API continues to function, but is more cumbersome than necessary. There appears to be no compelling case that the API is a risk through misuse. However, if updating our reference to the C23 library removes the `ATOMIC_VAR_INT` macro, we might want to consider this removal for C++26.

Additionally, the volatile qualified member functions of the `atomic` class template were deprecated for C++20 by paper [P1831R1]. Their removal should be considered as part of [P2866] proposing removal of deprecated volatile operations.

6.30.1 Initial LEWG Review: Kona, 2023/11/07

TBD

6.30.2 LEWG electronic poll : 2023/12/20 – 2023/01/10

Poll taken: [P3053R0]

Poll results: [P3054R0]

Conclusion: Forward to LWG

7 Acknowledgements

Thanks to Michael Park for the pandoc-based framework used to generate this paper from extended markdown source.

Special thanks for Matt Godbolt for Compiler Explorer, that made it incredibly simple to test deprecated code samples across a variety of compilers, ancient and modern. This paper would be much less informed without the rapid testing it enabled.

§ 8 References

[CWG2521] Jim X. 2022-01-07. User-defined literals and reserved identifiers.
https://wg21.link/cwg2521

[LWG3036] Casey Carter. polymorphic_allocator::destroy is extraneous.
https://wg21.link/lwg3036

[LWG3170] Billy O'Neal III. is_always_equal added to std::allocator makes the standard library treat derived types as always equal.
https://wg21.link/lwg3170

[LWG3767] Victor Zverovich. codecvt<charN_t, char8_t, mbstate_t> incorrectly added to locale.
https://wg21.link/lwg3767

[LWG3840] Daniel Krügler. filesystem::u8path should be undeprecated.
https://wg21.link/lwg3840

[N2007] P.J. Plauger. 2006-04-15. Proposed Library Additions for Code Conversion.
https://wg21.link/n2007

[N3203] Jens Maurer. 2010-11-11. Tightening the conditions for generating implicit moves.
https://wg21.link/n3203

[N4950] Thomas Köppe. 2023-05-10. Working Draft, Standard for Programming Language C++.
https://wg21.link/n4950

[P0174R2] Alisdair Meredith. 2016-06-23. Deprecating Vestigial Library Parts in C++17.
https://wg21.link/p0174r2

[P0218R1] Beman Dawes. 2016-03-05. Adopt File System TS for C++17.
https://wg21.link/p0218r1

[P0270R3] Hans Boehm. 2017-02-02. Removing C dependencies from signal handler wording.
https://wg21.link/p0270r3

[P0386R2] Hal Finkel, Richard Smith. 2016-06-24. Inline Variables.
https://wg21.link/p0386r2

[P0482R6] Tom Honermann. 2018-11-09. char8_t: A type for UTF-8 characters and strings (Revision 6).
https://wg21.link/p0482r6

[P0618R0] Alisdair Meredith. 2017-03-02. Deprecating <codecvt>.
https://wg21.link/p0618r0

[P0718R2] Alisdair Meredith. 2017-11-10. Revising atomic_shared_ptr for C++20.
https://wg21.link/p0718r2

[P0767R1] Jens Maurer. 2017-11-10. Deprecate POD.
https://wg21.link/p0767r1

[P0768R1] Walter E. Brown. 2017-11-10. Library Support for the Spaceship (Comparison) Operator.
https://wg21.link/p0768r1

[P0788R3] Walter E. Brown. 2018-06-07. Standard Library Specification in a Concepts and Contracts World.
https://wg21.link/p0788r3

[P0806R2] Thomas Köppe. 2018-06-04. Deprecate implicit capture of this via [=].
https://wg21.link/p0806r2

[P0883R2] Nicolai Josuttis. 2019-11-08. Fixing Atomic Initialization.
https://wg21.link/p0883r2

[P0896R4] Eric Niebler, Casey Carter, Christopher Di Bella. 2018-11-09. The One Ranges Proposal.
https://wg21.link/p0896r4

[P0966R1] Mark Zeren, Andrew Luo. 2018-02-08. string::reserve Should Not Shrink.
https://wg21.link/p0966r1

[P1120R0] Richard Smith. 2018-06-08. Consistency improvements for <=> and other comparison operators.
https://wg21.link/p1120r0

[P1152R4] JF Bastien. 2019-07-22. Deprecating volatile.
https://wg21.link/p1152r4

[P1252R2] Casey Carter. 2019-02-22. Ranges Design Cleanup.
https://wg21.link/p1252r2

[P1413R3] CJ Johnson. 2021-11-22. Deprecate std::aligned_storage and std::aligned_union.
https://wg21.link/p1413r3

[P1787R6] S. Davis Herring. 2020-10-28. Declarations and where to find them.
https://wg21.link/p1787r6

[P1815R2] S. Davis Herring. 2020-02-14. Translation-unit-local entities.
https://wg21.link/p1815r2

[P1831R1] JF Bastien. 2020-02-12. deprecating volatile: library.
https://wg21.link/p1831r1

[P2327R1] Paul M. Bendixen, Jens Maurer, Arthur O'Dwyer, Ben Saks. 2021-10-04. De-deprecating volatile compound operations.
https://wg21.link/p2327r1
[P2340R1] Thomas Köppe. 2021-06-11. Clarifying the status of the "C headers."
https://wg21.link/p2340r1
[P2614R2] Matthias Kretz. 2022-11-08. Deprecate numeric_limits::has_denorm.
https://wg21.link/p2614r2
[P2626R0] Corentin Jabot. 2022-08-09. charN_t incremental adoption: Casting pointers of UTF character types.
https://wg21.link/p2626r0
[P2790R0] Jonathan Wakely. 2023-02-13. C++ Standard Library Immediate Issues to be moved in Issaquah, Feb. 2023.
https://wg21.link/p2790r0
[P2864R2] Alisdair Meredith. 2023-11-11. Remove Deprecated Arithmetic Conversion on Enumerations From C++26.
https://wg21.link/p2864r2
[P2865] Alisdair Meredith. Remove Deprecated Array Comparisons from C++26.
https://wg21.link/p2865
[P2866] Alisdair Meredith. Remove Deprecated Volatile Features From C++26.
https://wg21.link/p2866
[P2867R2] Alisdair Meredith. Remove Deprecated strstreams From C++26.
https://wg21.link/p2867r2
[P2868R3] Alisdair Meredith. 2023-11-11. Remove Deprecated `std::allocator` Typedef From C++26.
https://wg21.link/p2868r3
[P2869R4] Alisdair Meredith. Remove Deprecated `shared_ptr` Atomic Access APIs From C++26.
https://wg21.link/p2869r4
[P2870R3] Alisdair Meredith. 2023-11-11. Remove `basic_string::reserve()` From C++26.
https://wg21.link/p2870r3
[P2871R3] Alisdair Meredith. 2023-11-11. Remove Deprecated Unicode Conversion Facets From C++26.
https://wg21.link/p2871r3
[P2872R3] Alisdair Meredith. Remove `wstring_convert` From C++26.
https://wg21.link/p2872r3
[P2873] Alisdair Meredith. Remove Deprecated Locale Category Facets For Unicode from C++26.
https://wg21.link/p2873
[P2874R1] Alisdair Meredith. 2023-06-12. Mandating Annex D.
https://wg21.link/p2874r1
[P2875R4] Alisdair Meredith. Undeprecate `polymorphic_allocator::destroy` For C++26.
https://wg21.link/p2875R4
[P2972R0] Inbal Levi, Ben Craig, Fabio Fracassi, Corentin Jabot, Nevin Liber, Billy Baker. 2023-09-18. 2023-09 Library Evolution Polls.
https://wg21.link/p2972r0
[P2984R0] Alisdair Meredith. 2023-10-15. Reconsider Redeclaring static constexpr Data Members.
https://wg21.link/p2984r0
[P3020R0] Inbal Levi, Fabio Fracassi, Ben Craig, Billy Baker, Nevin Liber, Corentin Jabot. 2023-10-15. 2023-09 Library Evolution Poll Outcomes.
https://wg21.link/p3020r0
[P3039R0] David Stone. 2023-12-15. Automatically Generate `operator->`.
https://wg21.link/p3039r0
[P3047] Alisdair Meredith. Remove Namespace relops From C++26.
https://wg21.link/p3047r0
[P3053R0] Inbal Levi, Fabio Fracassi, Ben Craig, Nevin Liber, Billy Baker, Corentin Jabot. 2023-12-15. 2023-12 Library Evolution Polls.
https://wg21.link/p3053r0
[P3054R0] Inbal Levi, Fabio Fracassi, Ben Craig, Billy Baker, Nevin Liber, Corentin Jabot. 2024-01-13. 2023-12 Library Evolution Poll Outcomes.
https://wg21.link/p3054r0